

Program Testing

Verification versus Validation

- Verification is the process of determining:
 - Whether output of one phase of development conforms to its previous phase.
- Validation is the process of determining:
 - Whether a fully developed system conforms to its SRS document.
- Verification is concerned with phase containment of errors,
 - Whereas the aim of validation is that the final product be error free.

Testing

- The aim of testing is to identify all defects in a software product.
- However, in practice even after thorough testing:
 - one cannot guarantee that the software is error-free.
- The input data domain of most software products is very large:
 - It is not practical to test the software exhaustively with each input data value.
- Testing does however expose many errors:
 - Testing provides a practical way of reducing defects in a system
 - Increases the users' confidence in a developed system.

Testing

- Testing is an important development phase:
 - requires the maximum effort among all development phases.
- In a typical development organization:
 - maximum number of software engineers can be found to be engaged in testing activities.
- Many engineers have the wrong impression:
 - testing is a secondary activity
 - it is intellectually not as stimulating as the other development activities, etc.

Testing

- Testing a software product is in fact:
 - as much challenging as initial development activities such as specification, design, and coding.
- Also, testing involves a lot of creative thinking.
- Software products are tested at three levels:
 - Unit testing
 - Integration testing
 - System testing

Unit testing

- During unit testing, modules are tested in isolation:
 - If all modules were to be tested together:
 - it may not be easy to determine which module has the error.
- Unit testing reduces debugging effort several folds.
 - Programmers carry out unit testing immediately after they complete the coding of a module.

Integration testing

- After different modules of a system have been coded and unit tested:
 - modules are integrated in steps according to an integration plan
 - partially integrated system is tested at each integration step.

System Testing

- System testing involves:
 - validating a fully developed system against its requirements.

Integration Testing

- Develop the integration plan by examining the structure chart :
 - big bang approach
 - top-down approach
 - bottom-up approach
 - mixed approach

Big Bang Integration Testing

- Big bang approach is the simplest integration testing approach:
 - all the modules are simply put together and tested.
 - this technique is used only for very small systems.
- Main problems with this approach:
 - If an error is found:
 - It is very difficult to localize the error
 - The error may potentially belong to any of the modules being integrated.
 - Debugging errors found during big bang integration testing are very expensive to fix.

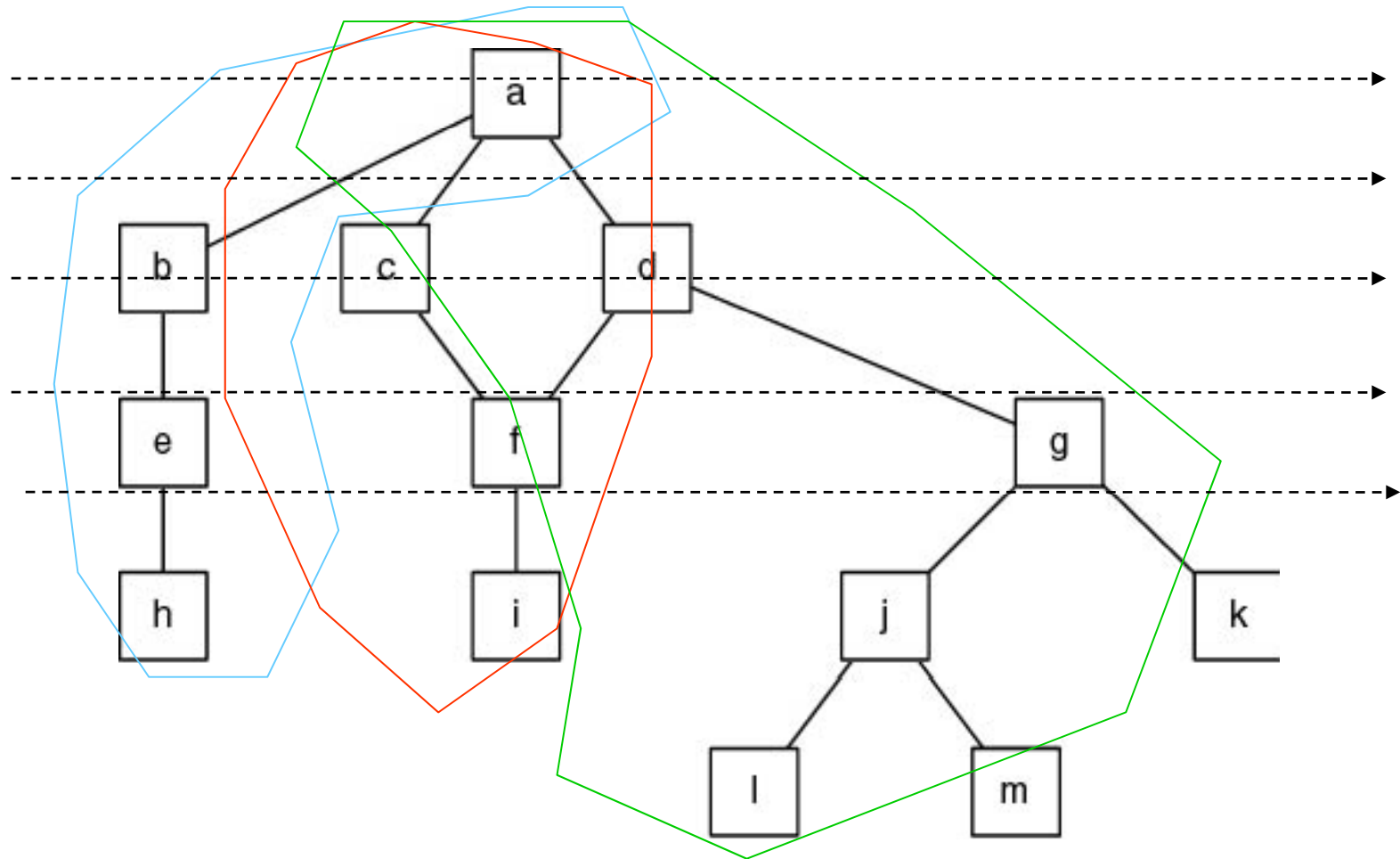
Bottom-up Integration Testing

- **Integrate and test the bottom level modules first.**
- **A disadvantage of bottom-up testing:**
 - **when the system is made up of a large number of small subsystems.**
 - **This extreme case corresponds to the big bang approach.**

Top-down Integration Testing

- **Top-down integration testing starts with the main routine:**
 - **and one or two subordinate routines in the system.**
- **After the top-level 'skeleton' has been tested:**
 - **immediate subordinate modules of the 'skeleton' are combined with it and tested.**

Top-Down



Integration Testing

- **In top-down approach:**
 - **testing waits till all top-level modules are coded and unit tested.**
- **In bottom-up approach:**
 - **testing can start only after bottom level modules are ready.**

Mixed Integration Testing

- **Mixed (or sandwiched) integration testing:**
 - **uses both top-down and bottom-up testing approaches.**
 - **Most common approach**

Phased vs. Incremental Integration Testing

- Integration can be incremental or phased.
- In incremental integration testing,
 - only one new module is added to the partial system each time.
- In phased integration,
 - A group of related modules are added to the partially integrated system each time.
- Big-bang testing:
 - A degenerate case of the phased integration testing.
- Phased integration requires less number of integration steps:
 - compared to the incremental integration approach.
- However, when failures are detected,
 - it is easier to debug if using incremental testing
 - since errors are very likely to be in the newly integrated module.

Summary of Integration Approaches

Approach	Strengths	Weaknesses
Implementation then integration	—	No fault isolation Major design faults show up late
Top-down implementation and integration	Fault isolation Major design faults show up early	Potentially reusable modules are not adequately tested
Bottom-up implementation and integration	Fault isolation Potentially reusable modules are adequately tested	Major design faults show up late
Sandwich implementation and integration	Fault isolation Major design faults show up early Potentially reusable modules are adequately tested	—

System Testing

- There are three main kinds of system testing:
 - Alpha Testing
 - Beta Testing
 - Acceptance Testing
- During system testing:
 - **Functional requirements** are validated through functional tests.
 - **Non-functional requirements** validated through performance tests.

Alpha Testing

- **System testing is carried out by the test team within the developing organization.**

Beta Testing

- **System testing performed by a select group of friendly customers.**

Acceptance Testing

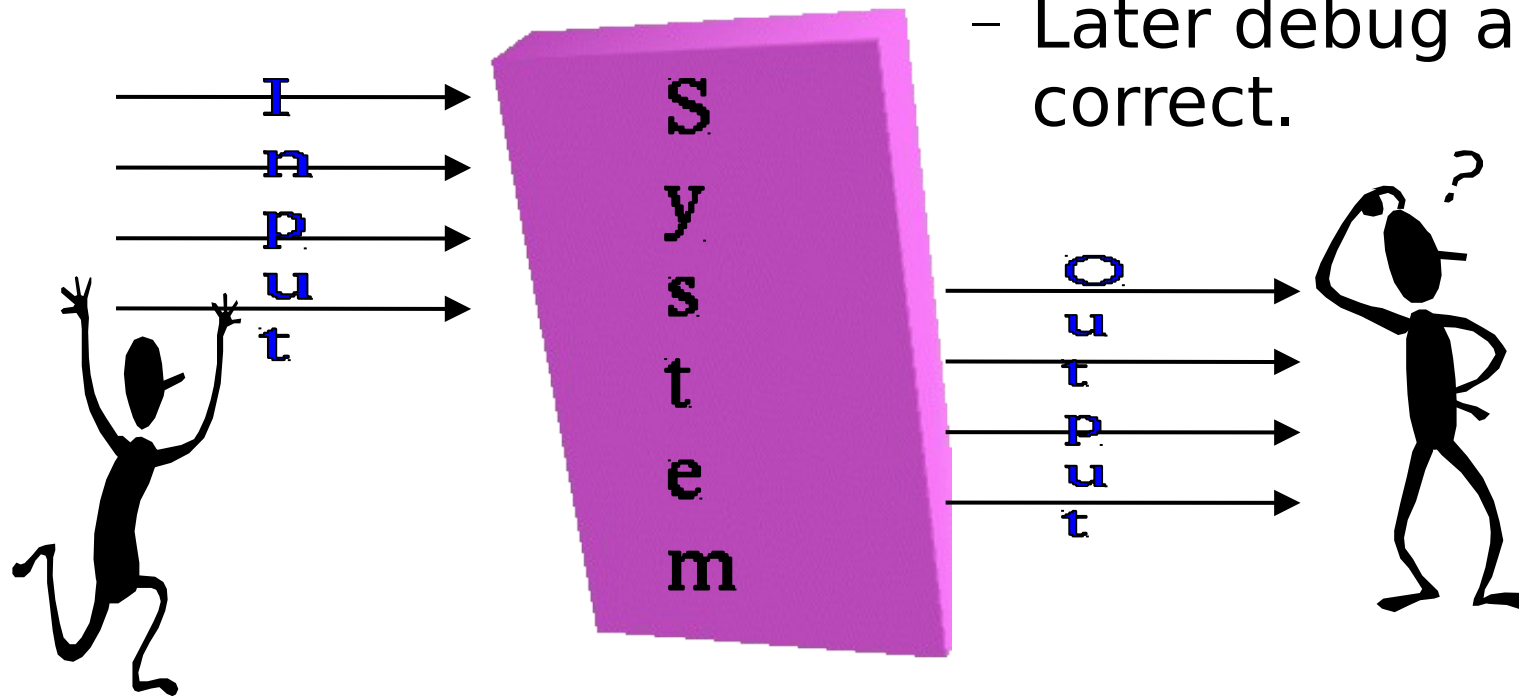
- **System testing performed by the customer himself:**
 - **to determine whether the system should be accepted or rejected.**

Acceptance Testing

- Test software with the client's environment and real data
- Performed by the SQA team in the presence of client representatives, or an independent SQA team hired by the client

How Do You Test a Program?

- Input test data to the program.
- Observe the output:
 - Check if the program behaved as expected.
- If the program does not behave as expected:
 - Note the conditions under which it failed.



Overview of Testing Activities

- **Test Suite Design**
- **Run test cases and observe results to detect failures.**
- **Debug to locate errors**
- **Correct errors.**

Test cases and Test suites

- Test a software using a set of carefully designed test cases:
 - The set of all test cases is called the test suite
- A test case is a triplet [I,S,O]
 - I is the data to be input to the system,
 - S is the state of the system at which the data will be input,
 - O is the expected output of the system.

Design of Test Cases

- Exhaustive testing of any non-trivial system is impractical:
 - Input data domain is extremely large.
- Design an optimal test suite:
 - Of reasonable size and
 - Uncovers as many errors as possible.
- If test cases are selected randomly:
 - Many test cases would not contribute to the significance of the test suite,
 - Would not detect errors not already being detected by other test cases in the suite.
- Number of test cases in a randomly selected test suite:
 - Not an indication of effectiveness of testing.

Design of Test Cases

- Testing a system using a large number of randomly selected test cases:
 - Does not mean that many errors in the system will be uncovered.
- Consider following example:
 - Find the maximum of two integers x and y .
- The code has a simple programming error:
- $\text{If } (x > y) \text{ max} = x;$
 $\text{else max} = x;$
- Test suite $\{(x=3, y=2); (x=2, y=3)\}$ can detect the error,
- A larger test suite $\{(x=3, y=2); (x=4, y=3); (x=5, y=1)\}$ does not detect the error.
- Systematic approaches are required to design an optimal test suite:
 - Each test case in the suite should detect different errors.

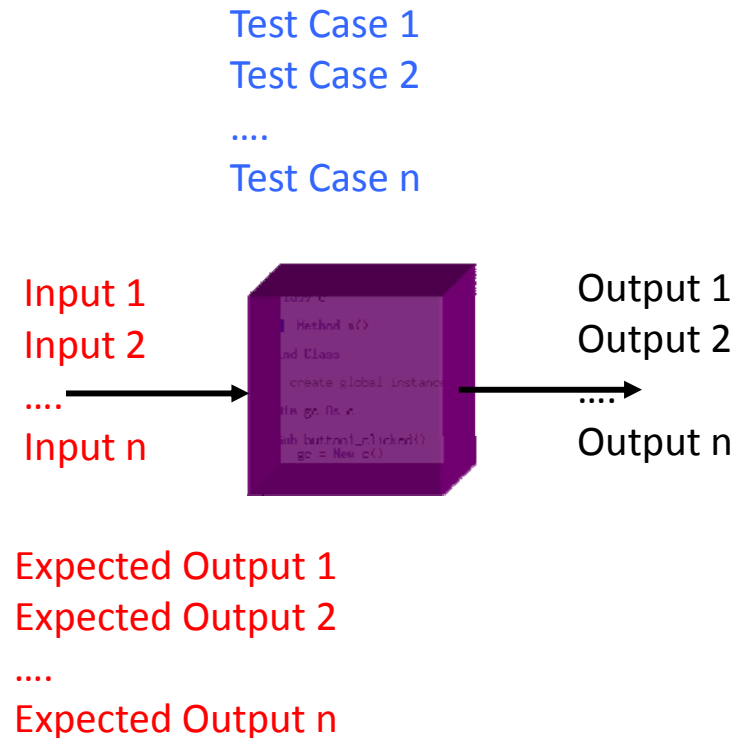
Methods to Design Test Cases

- Two main approaches to testing – Black-box and White-box (or glass box)
- Black Box approach
 - Test cases are designed using only functional specs. of the software, without any knowledge of the internal structure of the software.
 - For this reason, black-box testing is also known as Functional testing.
 - Two main approaches to design black box test cases:
 - Equivalence class partitioning
 - Boundary value analysis
- White Box approach
 - Designing white-box test cases requires knowledge about the internal structure of software.
 - White-box testing is also called Structural testing.
 - Several popular methodologies to design white-box test cases:
 - Statement coverage
 - Branch coverage
 - Path coverage
 - Condition coverage
 - Mutation testing
 - Data flow-based testing

Black Box Testing Techniques

Black Box Testing

- Behavioral
- Functional
- Data-driven
- Input/output-driven



Equivalence Class Partitioning

- Input values to a program are partitioned into **equivalence classes**.
- Partitioning is done such that:
 - Program behaves in similar ways to every input value belonging to an equivalence class.

Why Define Equivalence Classes?

- Test the code with just one representative value from each equivalence class:
 - As good as testing using any other values from the equivalence classes.
- How do you determine the equivalence classes?
 - Examine the input data.
 - Few general guidelines for determining the equivalence classes can be given

Equivalence Class Partitioning

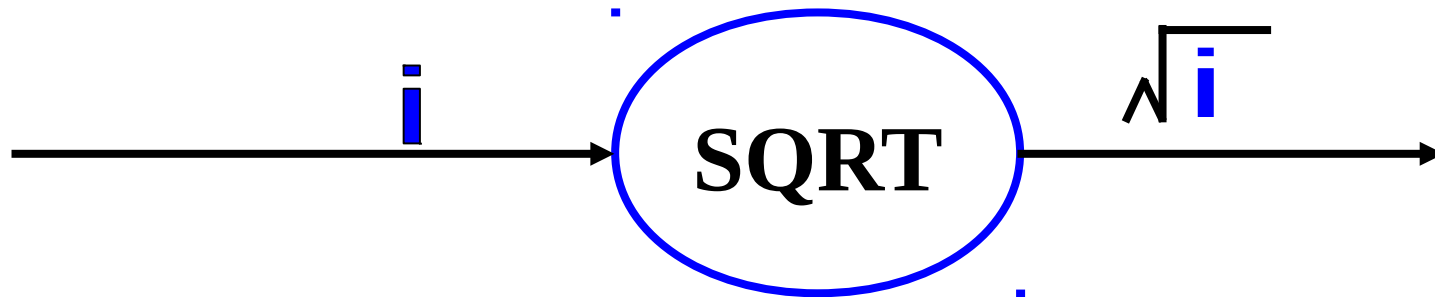
- If the input data to the program is specified by a **range of values**:
 - e.g. numbers between 1 to 5000.
 - One valid and two invalid equivalence classes are defined.



- If input is an enumerated set of values:
 - e.g. {a,b,c}
 - One equivalence class for valid input values.
 - Another equivalence class for invalid input values should be defined.

Example

- A program reads an input value in the range of 1 and 5000:
 - Computes the square root of the input number



- There are three equivalence classes:
 - The set of negative integers,
 - Set of integers in the range of 1 and 5000,
 - Integers larger than 5000.



Type of Equivalence Class

- A range of values
- A set of values
 - The input must be letter
- A specific value
 - The response must be followed by a # sign

How to Perform Equivalence Testing

- For each range (L, U)
 - Select five test cases: less than L, equal to L, greater than L but less than U, equal to U, and greater than U
- For each set S
 - Select two test cases: a member of S and a non-member of S
- For each precise value P
 - Select two test cases: P and anything else

Conditions of Triangle

- If you have three number, a , b , and c , which are the lengths of the side of a triangle, the conditions in triangle problem might be
 - $a = b$ or $a = c$ or $b = c$
 - $a = b$ and $b = c$
 - $a < b + c$ and $b < a + c$ and $c < a + b$
 - $a > 0$ and $b > 0$ and $c > 0$

Triangle Problem

- Equivalence classes
 - Scalene
 - Isosceles
 - Equilateral
 - Not a triangle
 - Bad inputs

Triangle Problem

- Equivalence classes
 - Scalene
 - Increasing size - (3, 4, 5)
 - Decreasing size - (5, 4, 3)
 - Largest as second - (4, 5, 3)
 - Isosceles
 - $a = b$ & other side larger – (5, 5, 8)
 - $a = c$ & other side larger – (5, 8, 5)
 - $b = c$ & other side larger – (8, 5, 5)
 - $a = b$ & other side smaller – (8, 8, 5)
 - $a = c$ & other side smaller – (8, 5, 8)
 - $b = c$ & other side smaller – (5, 8, 8)

Triangle Problem (cont'd)

- Equilateral
 - All sides equal (5, 5, 5)
- Not a triangle
 - Largest first (6, 4, 2)
 - Largest second (4, 6, 2)
 - Largest third (1, 2, 3)
- Bad inputs
 - One bad input (-1, 2, 4)
 - Two bad inputs (3, -2, -5)
 - Three bad inputs (0, 0, 0)

Boundary Value Analysis

- Some typical programming errors occur:
 - At boundaries of equivalence classes
 - Might be purely due to psychological factors.
- Programmers often fail to see:
 - Special processing required at the boundaries of equivalence classes.
- Programmers may improperly use `<` instead of `<=`
- Boundary value analysis:
 - Select test cases at the boundaries of different equivalence classes.

Example

- For a function that computes the square root of an integer in the range of 1 and 5000:
 - Test cases must include the values: {0,1,5000,5001}.

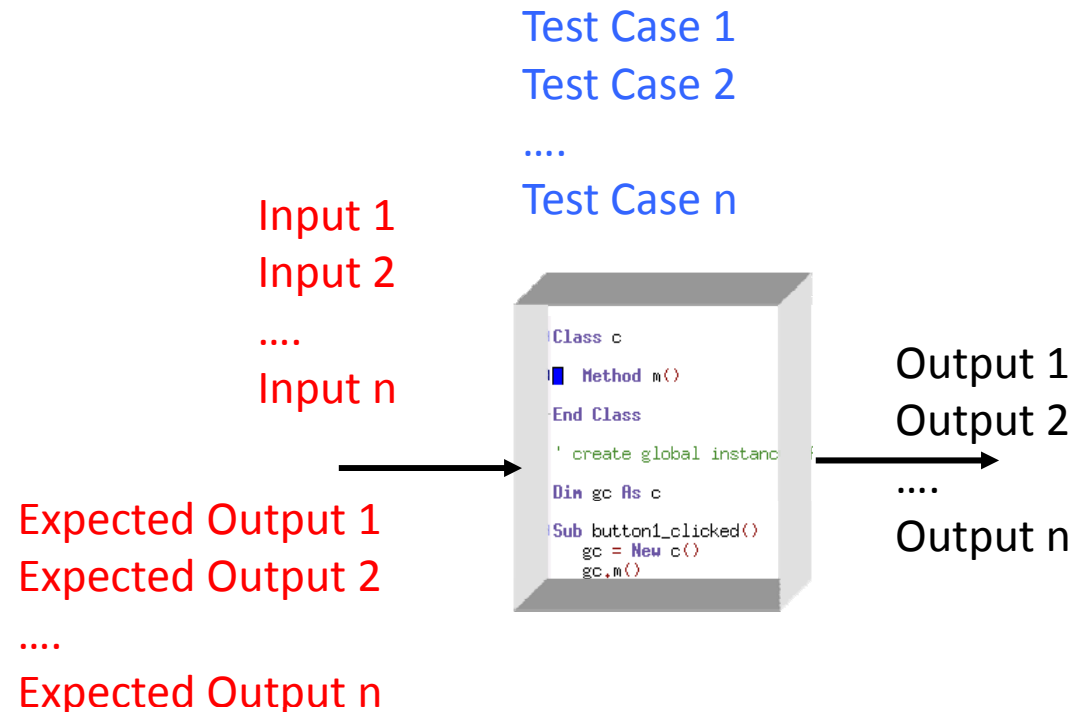
Coverage-Based Testing Versus Fault-Based Testing

- Idea behind coverage-based testing:
 - Design test cases so that certain program elements are executed (or covered).
 - Example: Statement coverage, Path coverage, etc.
- Idea behind fault-based testing:
 - Design test cases that focus on discovering certain types of faults.
 - Example: Mutation testing.

White Box Testing Techniques

White Box Testing

- White box testing
- Glass box testing
- Structural testing
- Logic-driven testing
- Path-oriented testing
- Testing to code



The most common form of testing to code requires that each path through the code be executed at least once.

Statement Coverage

- Statement coverage methodology:
 - Design test cases so that every statement in the program is executed at least once.
- The principal idea:
 - Unless a statement is executed,
 - We have no way of knowing if an error exists in that statement.
- Observing that a statement behaves properly for one input value:
 - No guarantee that it will behave correctly for all input values.

Branch Coverage

- Test cases are designed such that:
 - Different branch conditions
 - Given true and false values in turn.
- Branch testing guarantees statement coverage:
 - A stronger testing compared to the statement coverage-based testing.
- Stronger Testing
 - Test cases are a superset of a weaker testing.
 - A stronger testing covers at least all the elements of the elements covered by a weaker testing.

Condition Coverage

- Test cases are designed such that:
 - Each component of a composite conditional expression
 - Given both true and false values.
- Example
 - Consider the conditional expression $((c1.and.c2).or.c3)$:
 - Each of $c1$, $c2$, and $c3$ are exercised at least once, i.e. given true and false values.
- Consider a boolean expression having n components:
 - For condition coverage we require 2^n test cases.
- Condition coverage-based testing technique:
 - Practical only if n (the number of component conditions) is small.

Path Coverage

- Design test cases such that:
 - All linearly independent paths in the program are executed at least once.
- Defined in terms of
 - Control flow graph (CFG) of a program.

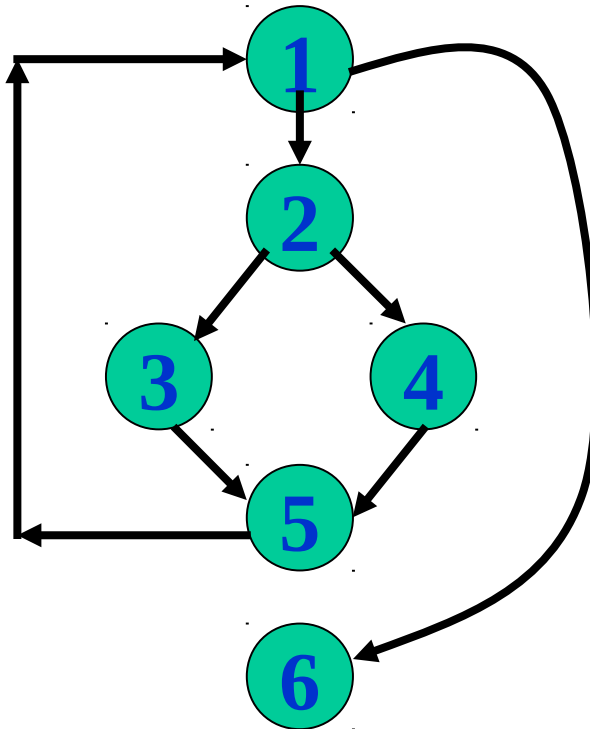
Path Coverage based Testing

- To understand the path coverage-based testing:
 - we need to learn how to draw control flow graph of a program.
- A control flow graph (CFG) describes:
 - the sequence in which different instructions of a program get executed.
 - the way control flows through the program.

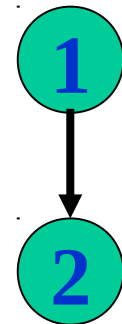
How to Draw Control Flow Graph?

- Number all the statements of a program.
- Numbered statements:
 - Represent nodes of the control flow graph.
- An edge from one node to another node exists:
 - If execution of the statement representing the first node
 - Can result in transfer of control to the other node.

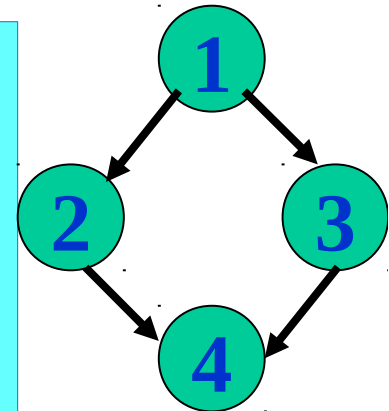
```
int f1(int x,int y){  
  1 while (x != y){  
    2   if (x>y) then  
    3     x=x-y;  
    4   else y=y-x;  
    5 }  
  6 return x;  
}
```



- Sequence:
 - 1 a=5;
 - 2 b=a*b-1;



- Selection:
 - 1 if(a>b) then
 - 2 c=3;
 - 3 else c=5;
 - 4 c=c*c;



- Iteration:
 - 1 while(a>b){
 - 2 b=b*a; b=b-1;}
 - 3 c=b+d;

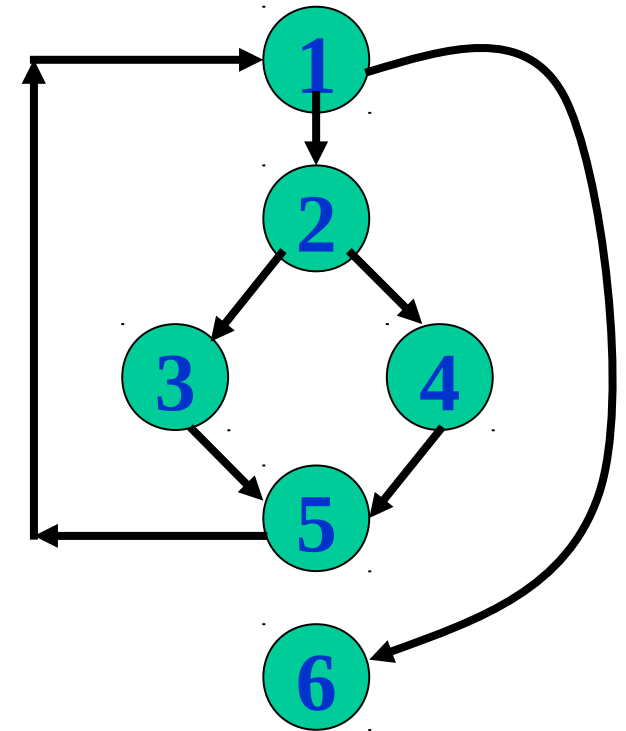
Path

- A path through a program:
 - A node and edge sequence from the starting node to a terminal node of the control flow graph (CFG).
 - There may be several terminal nodes for program.
- Linearly Independent Path
 - Any path through the program **Introducing at least one new edge, That is not included in any other independent paths.**
 - It is straight forward to identify linearly independent paths of **simple programs.**
 - For **complicated programs**, It is not so easy to determine the number of independent paths.

McCabe's Cyclomatic Metric

- An upper bound:
 - For the number of linearly independent paths of a program
- Provides a practical way of determining:
 - The maximum number of linearly independent paths in a program.

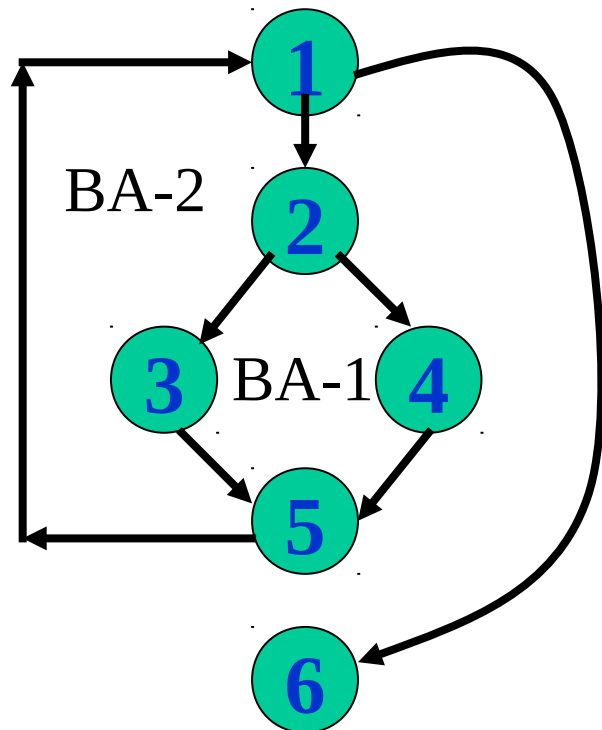
- Given a control flow graph G , cyclomatic complexity $V(G)$:
 - $V(G) = E - N + 2$
 - N is the number of nodes in G
 - E is the number of edges in G



$$\text{Cyclomatic complexity} = 7 - 6 + 2 = 3$$

Cyclomatic Complexity

- Another way of computing cyclomatic complexity:
 - inspect control flow graph
 - determine number of bounded areas in the graph
- $V(G) = \text{Total number of bounded areas} + 1$
 - Any region enclosed by a nodes and edge sequence.



- From a visual examination of the CFG:
 - the number of bounded areas is 2.
 - cyclomatic complexity = $2+1=3$.

Cyclomatic complexity

- McCabe's metric provides:
 - A quantitative measure of testing difficulty and the ultimate reliability
- Intuitively,
 - Number of bounded areas increases with the number of decision nodes and loops.
- The cyclomatic complexity of a program provides:
 - A lower bound on number of test cases to be designed to guarantee coverage of all linearly independent paths.
- Defines the number of independent paths in a program.
- Provides a lower bound:
 - for the number of test cases for path coverage.
- Cyclomatic complexity of a program:
 - Also indicates the psychological complexity of a program.
 - Difficulty level of understanding the program.
- From maintenance perspective,
 - Good software development organizations restrict cyclomatic complexity of functions to a maximum of ten or so.

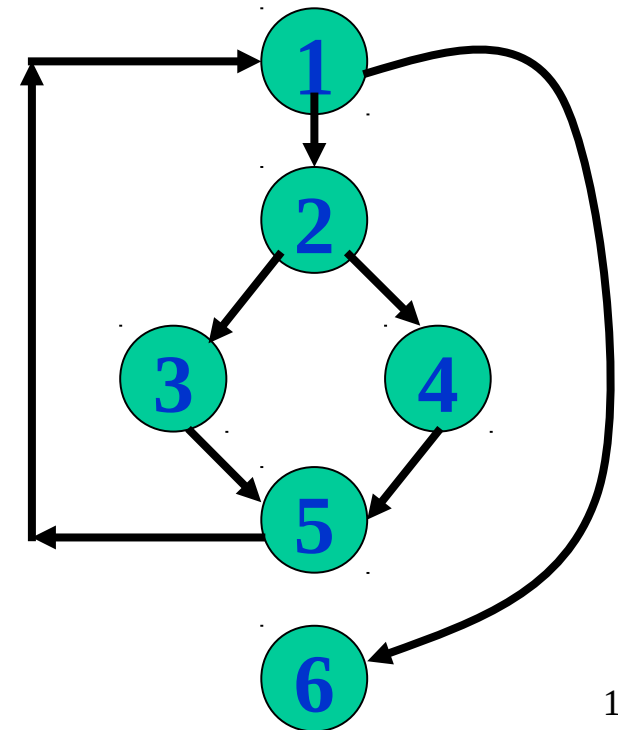
Derivation of Test Cases

- Let us discuss the steps:
 - to derive path coverage-based test cases of a program.
- Draw control flow graph.
- Determine $V(G)$.
- Determine the set of linearly independent paths.
- Prepare test cases:
 - to force execution along each path.

Derivation of Test Cases

- **Number of independent paths: 3**
 - **1,6** **test case (x=1, y=1)**
 - **1,2,3,5,1,6** **test case(x=2, y=1)**
 - **1,2,4,5,1,6** **test case(x=1, y=2)**

```
• int f1(int x,int y){  
• 1 while (x != y){  
• 2   if (x>y) then  
• 3     x=x-y;  
• 4   else y=y-x;  
• 5 }  
• 6 return x;    }
```



Other Testing Techniques

Stress Testing

- Stress testing (aka endurance testing):
 - impose abnormal input to stress the capabilities of the software.
 - Input data volume, input data rate, processing time, utilization of memory, etc. are tested beyond the designed capacity.
- Evaluates system performance - when stressed for short periods of time.
- Stress tests are black box tests:
 - Designed to impose a range of abnormal and even illegal input conditions
 - So as to stress the capabilities of the software.

Stress Testing

- If the requirements is to handle a specified number of users, or devices:
 - Stress testing evaluates system performance when all users or devices are busy simultaneously.
- If an operating system is supposed to support 15 multiprogrammed jobs,
 - The system is stressed by attempting to run 15 or more jobs simultaneously.
- A real-time system might be tested
 - To determine the effect of simultaneous arrival of several high-priority interrupts.

Stress Testing

- Stress testing usually involves an element of time or size,
 - Such as the number of records transferred per unit time,
 - The maximum number of users active at any time, input data size, etc.
- Therefore stress testing may not be applicable to many types of systems.

Volume Testing

- Addresses handling large amounts of data in the system:
 - Whether data structures (e.g. queues, stacks, arrays, etc.) are large enough to handle all possible situations.
 - Fields, records, and files are stressed to check if their size can accommodate all possible data volumes.

Configuration Testing

- Analyze system behavior:
 - in various hardware and software configurations specified in the requirements
 - sometimes systems are built in various configurations for different users
 - for instance, a minimal system may serve a single user,
 - other configurations for additional users.

Compatibility Testing

- These tests are needed when the system interfaces with other systems:
 - Check whether the interface functions as required.
- If a system is to communicate with a large database system to retrieve information:
 - A compatibility test examines speed and accuracy of retrieval.

Recovery Testing

- These tests check response to:
 - Presence of faults or to the loss of data, power, devices, or services
 - Subject system to loss of resources
 - Check if the system recovers properly.

Maintenance Testing

- Diagnostic tools and procedures:
 - help find source of problems.
 - It may be required to supply
 - memory maps
 - diagnostic programs
 - traces of transactions,
 - circuit diagrams, etc.
- Verify that:
 - all required artifacts for maintenance exist
 - they function properly

Documentation tests

- Check that required documents exist and are consistent:
 - user guides,
 - maintenance guides,
 - technical documents
- Sometimes requirements specify:
 - Format and audience of specific documents
 - Documents are evaluated for compliance

Usability tests

- All aspects of user interfaces are tested:
 - Display screens
 - messages
 - report formats
 - navigation and selection problems

Environmental test

- These tests check the system's ability to perform at the installation site.
- Requirements might include tolerance for
 - heat
 - humidity
 - chemical presence
 - portability
 - electrical or magnetic fields
 - disruption of power, etc.

Regression Testing

- Does not belong to either unit test, integration test, or system test.
 - In stead, it is a separate dimension to these three forms of testing.
- Regression testing is the running of test suite:
 - after each change to the system after each bug fix.
 - Ensures that no new bug has been introduced due to the change or the bug fix.
- Regression tests assure:
 - the new system's performance is at least as good as the old system.
 - Always used during incremental system development.

Test Summary Report

- Generated towards the end of testing phase.
- Covers each subsystem:
 - A summary of tests which have been applied to the subsystem.
- Specifies:
 - how many tests have been applied to a subsystem,
 - how many tests have been successful,
 - how many have been unsuccessful, and the degree to which they have been unsuccessful,
 - e.g. whether a test was an outright failure
 - or whether some expected results of the test were actually observed.